

K L Deemed to be University

Department of Computer Science and Engineering | 2026–2027, Odd Semester

Database Systems Engineering and Distributed Backend Development (25CS1302E)

Practical 4

The Intelligent Catalog and API Blueprint

Name	Chakrika Panakala
Roll No.	2520090042
Course	25CS1302E
CO Mapping	CO2
Session	Practical 4

Aim

To enable pgvector in bookflow_db, add a VECTOR(3) embedding column to Books, insert mock theme vectors and find the three books closest to a search vector with the <-> (L2) and <=> (cosine) operators, and to design the OpenAPI contract for GET /v1/books/search with q and limit parameters, a 200 response schema and a 400 error envelope that follows REST constraints.

Program / Code

Part A – p4_vector.sql

```
-- Experiment 4 (Part A): Semantic search with pgvector

\c bookflow_db CREATE EXTENSION IF NOT EXISTS vector; ALTER TABLE Books ADD COLUMN embedding VECTOR(3); --
Mock 3-dimensional theme vectors -- dimension 1 = space / science fiction, 2 = computer science, 3 =
self-improvement UPDATE Books SET embedding = '[0.95, 0.10, 0.05]' WHERE isbn = '9780553418026'; -- The
Martian UPDATE Books SET embedding = '[0.85, 0.05, 0.10]' WHERE isbn = '9780441172719'; -- Dune UPDATE Books
SET embedding = '[0.92, 0.15, 0.08]' WHERE isbn = '9780593135204'; -- Project Hail Mary UPDATE Books SET
embedding = '[0.05, 0.95, 0.10]' WHERE isbn = '9780132350884'; -- Clean Code UPDATE Books SET embedding =
'[0.02, 0.90, 0.05]' WHERE isbn = '9780078022159'; -- Database System Concepts UPDATE Books SET embedding =
'[0.03, 0.92, 0.04]' WHERE isbn = '9781449373320'; -- Designing Data-Intensive Applications UPDATE Books SET
embedding = '[0.10, 0.80, 0.20]' WHERE isbn = '9780135957059'; -- The Pragmatic Programmer UPDATE Books SET
embedding = '[0.02, 0.97, 0.03]' WHERE isbn = '9780262046305'; -- Introduction to Algorithms UPDATE Books SET
embedding = '[0.05, 0.10, 0.95]' WHERE isbn = '9780735211292'; -- Atomic Habits UPDATE Books SET embedding =
'[0.05, 0.20, 0.90]' WHERE isbn = '9781455586691'; -- Deep Work SELECT book_id, title, embedding FROM Books
ORDER BY book_id;
```

L2 (Euclidean) search – “Stories about space”

```
SELECT title, ROUND((embedding <-> '[0.9, 0.1, 0.1]')::NUMERIC, 4) AS l2_distance FROM Books ORDER BY
embedding <-> '[0.9, 0.1, 0.1]' LIMIT 3;
```

Cosine search – “Stories about space”

```
SELECT title, ROUND((embedding <=> '[0.9, 0.1, 0.1]')::NUMERIC, 4) AS cosine_distance, ROUND((1 - (embedding
<=> '[0.9, 0.1, 0.1]'))::NUMERIC, 4) AS cosine_similarity FROM Books ORDER BY embedding <=> '[0.9, 0.1, 0.1]'
LIMIT 3;
```

Second search – “How to write better software”

```
SELECT title, ROUND((embedding <=> '[0.1, 0.8, 0.2]')::NUMERIC, 4) AS cosine_distance FROM Books ORDER BY
embedding <=> '[0.1, 0.8, 0.2]' LIMIT 3;
```

Part B – openapi.yaml

```
openapi: 3.0.3 info: title: BookFlow Catalog API version: 1.0.0 description: Contract for the BookFlow catalog
search service. servers: - url: http://localhost:8000 description: Local development server paths:
```

/v1/books/search: get: summary: Search the catalog (keyword and semantic search) operationId: searchBooks tags: [Catalog] parameters: - name: q in: query required: true schema: type: string minLength: 2 maxLength: 200 - name: limit in: query required: false schema: type: integer minimum: 1 maximum: 50 default: 10 responses: '200': description: Matching books ordered by similarity (most similar first). content: application/json: schema: \$ref: '#/components/schemas/SearchResponse' '400': description: The request parameters are invalid. content: application/json: schema: \$ref: '#/components/schemas/ErrorEnvelope' /v1/books: post: summary: Create a new book (POST is used for creating resources) operationId: createBook tags: [Catalog] requestBody: required: true content: application/json: schema: \$ref: '#/components/schemas/BookCreate' responses: '201': description: Book created. content: application/json: schema: \$ref: '#/components/schemas/Book' '400': description: Validation failed. content: application/json: schema: \$ref: '#/components/schemas/ErrorEnvelope'

OpenAPI Schemas

components: schemas: Book: type: object required: [book_id, title, isbn] properties: book_id: { type: integer, example: 10 } title: { type: string, example: Project Hail Mary } isbn: { type: string, minLength: 13, maxLength: 13, example: '9780593135204' } published_year: { type: integer, maximum: 2026, example: 2021 } BookCreate: type: object required: [title, isbn] properties: title: { type: string, minLength: 1 } isbn: { type: string, minLength: 13, maxLength: 13 } published_year: { type: integer, maximum: 2026 } SearchResult: allOf: - \$ref: '#/components/schemas/Book' - type: object required: [similarity] properties: similarity: type: number format: float minimum: 0 maximum: 1 SearchResponse: type: object required: [query, count, results] properties: query: { type: string } count: { type: integer, minimum: 0 } results: type: array items: \$ref: '#/components/schemas/SearchResult' ErrorEnvelope: type: object required: [error] properties: error: type: object required: [code, message] properties: code: { type: string, example: INVALID_PARAMETER } message: { type: string } field: { type: string, nullable: true } timestamp: { type: string, format: date-time }

Output

Part A – pgvector execution

Embedding column: VECTOR(3) was added successfully and the ten mock vectors were updated.

Book ID	Title	Embedding
1	Clean Code	[0.05, 0.95, 0.1]
2	Database System Concepts	[0.02, 0.9, 0.05]
3	The Martian	[0.95, 0.1, 0.05]
7	Dune	[0.85, 0.05, 0.1]
8	Atomic Habits	[0.05, 0.1, 0.95]
9	Designing Data-Intensive Applications	[0.03, 0.92, 0.04]
10	Project Hail Mary	[0.92, 0.15, 0.08]
11	The Pragmatic Programmer	[0.1, 0.8, 0.2]
12	Deep Work	[0.05, 0.2, 0.9]
13	Introduction to Algorithms	[0.02, 0.97, 0.03]

L2 distance – search vector [0.9, 0.1, 0.1]

Title	L2 Distance
Project Hail Mary	0.0574
Dune	0.0707
The Martian	0.0707

Cosine distance / similarity – search vector [0.9, 0.1, 0.1]

Title	Cosine Distance	Cosine Similarity
Dune	0.0014	0.9986
Project Hail Mary	0.0016	0.9984
The Martian	0.0017	0.9983

Second cosine search – [0.1, 0.8, 0.2]

Title	Cosine Distance
The Pragmatic Programmer	0.0000
Clean Code	0.0121
Database System Concepts	0.0226

Part B – OpenAPI validation

Command: openapi-spec-validator openapi.yaml

Output: openapi.yaml: OK

REST constraint verification:

Requirement	Verification
GET /v1/books/search	Safe, read-only fetch; q required and limit 1–50, default 10
POST /v1/books	Creates a resource and returns 201 Created
200 OK	SearchResponse {query, count, results[]} ordered by similarity
400 Bad Request	ErrorEnvelope {error: {code, message, field, timestamp}}
Statelessness	No cookies or session parameters; each request is self-contained
Uniform interface	Resource nouns (/books), versioned path (/v1), JSON media type

Result

For the “stories about space” vector [0.9, 0.1, 0.1], both L2 and cosine distance returned Project Hail Mary, Dune and The Martian among the closest books. The software-oriented query returned The Pragmatic Programmer, Clean Code and Database System Concepts. The OpenAPI 3.0.3 contract validated without errors and follows statelessness and the appropriate HTTP verbs (GET for fetching and POST for creating).

Conclusion

The experiment successfully demonstrates semantic book search using pgvector embeddings and distance operators in PostgreSQL. The OpenAPI contract defines the catalog search endpoint, request parameters, response schemas and error handling while maintaining REST constraints.